

Осуществление интеграции программных модулей

Руководитель практики от организации: Зязин Сергей Валерьевич

Руководитель практики от колледжа: Седов Алексей Сергеевич

Год: 2026г.

ПРОИЗВОДСТВЕННАЯ
ПРАКТИКА

ПЕСТОВ АРСЕНИЙ НИКОЛАЕВИЧ

ПМ. 02

ХАРАКТЕРИСТИКА ОБЪЕКТА ПРАКТИКИ

Объектом практики является отдел социального развития администрации города Слободского.

Юридический адрес: ул. Советская, 86, Слободской, Кировская обл., 613150.

Специализация отдела: организация социальной поддержки граждан, назначение пособий, работа с льготными категориями, ведение базы получателей социальных услуг.

Структура отдела: руководитель отдела, несколько специалистов по направлениям (социальные выплаты, работа с пожилыми, поддержка семей с детьми), бухгалтерия, системный администратор. Всего около 10–15 сотрудников.

Состав программного и технического обеспечения, имеющегося на предприятии:

1. Операционная система: Windows 10 Pro.
2. 1С:Предприятие
3. Антивирус ClamAV
4. Дополнительное ПО: Total Commander, IrfanView, Firefox.
5. Техническое обеспечение: компьютеры (Intel Core i3/i5, 8 ГБ ОЗУ), принтеры.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Предметная область - процесс приёма, регистрации, обработки и контроля исполнения обращений граждан в администрации.

Основные бизнес-процессы:

- подача обращения гражданином с указанием темы, текста и (опционально) прикреплённых файлов;
- проверка и регистрация обращения оператором приёмной, классификацию по категориям;
- назначение ответственного исполнителя из структурного подразделения;
- исполнение обращения, изменение его статуса, комментирование;
- контроль сроков и формирование отчётности для руководства.

Выделены следующие категории пользователей:

- граждане (заявители),
- операторы приёмной,
- исполнители (сотрудники отделов),
- администратор системы.



Диаграмма вариантов использования

ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

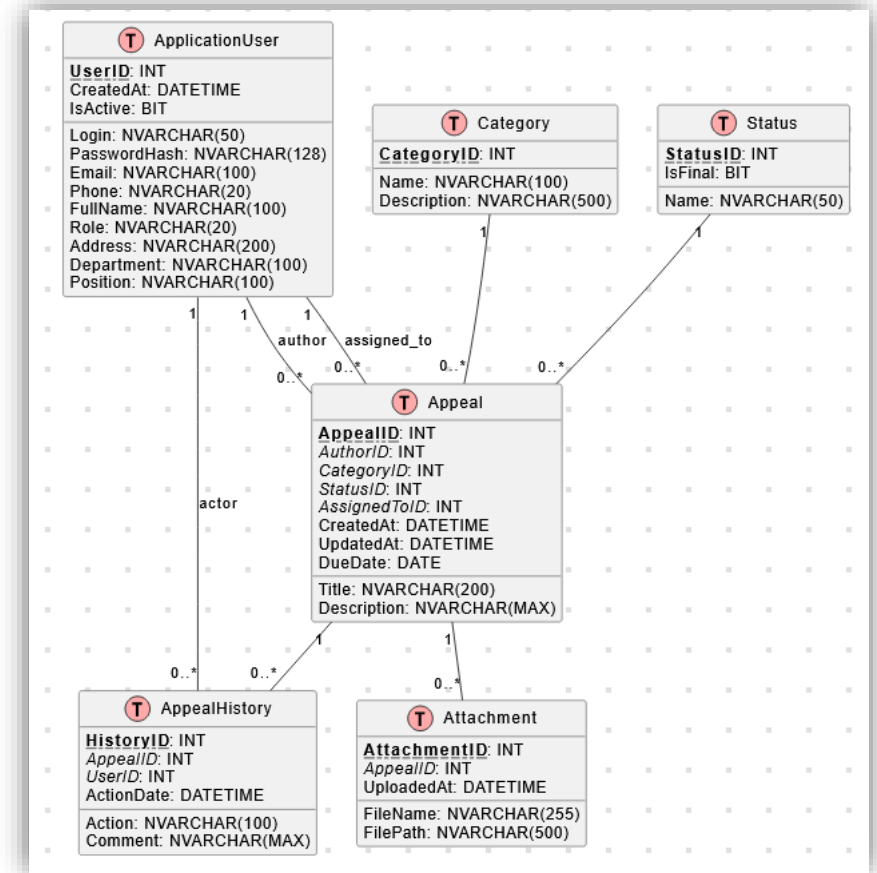
На основе ER-модели разработана реляционная база данных, нормализованная до третьей нормальной формы. Определены следующие таблицы:

- ApplicationUser – пользователи приложения (логин, пароль, роль, ФИО и др.);
- Appeal – обращения (автор, заголовок, описание, категория, статус, исполнитель);
- Category – категории обращений;
- Status – статусы обращений (с признаком конечности);
- AppealHistory – история действий (создание, смена статуса, комментарии);
- Attachment – прикрепленные файлы.

Связи между таблицами реализованы через внешние ключи с каскадным удалением для истории и вложений.

Физическая модель реализована в СУБД Microsoft SQL Server.

База данных заполнена свыше 2600 тестовыми записями.

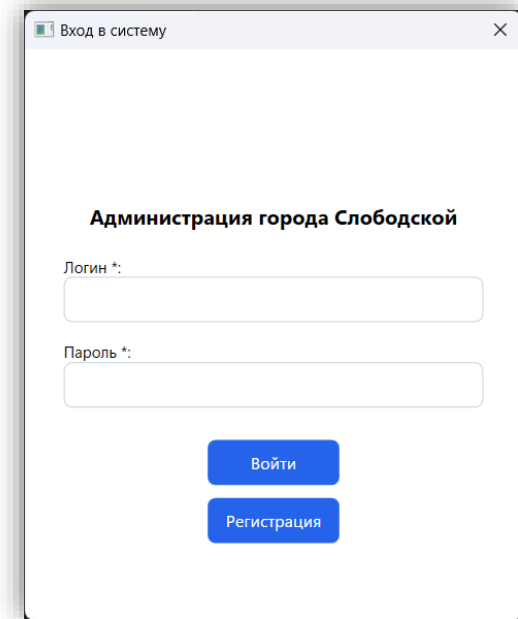


ER-диаграмма базы данных

РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

Серверная часть – предоставляет REST-эндпоинты для аутентификации, управления обращениями, пользователями, категориями, статусами и вложениями. Доступ к данным осуществляется через Entity Framework Core. Аутентификация выполняется по паре логин/пароль, передаваемых в заголовках запросов.

Клиентская часть – выполняет HTTP-запросы и десериализует ответы. Пользовательский интерфейс адаптируется под роль авторизованного пользователя.



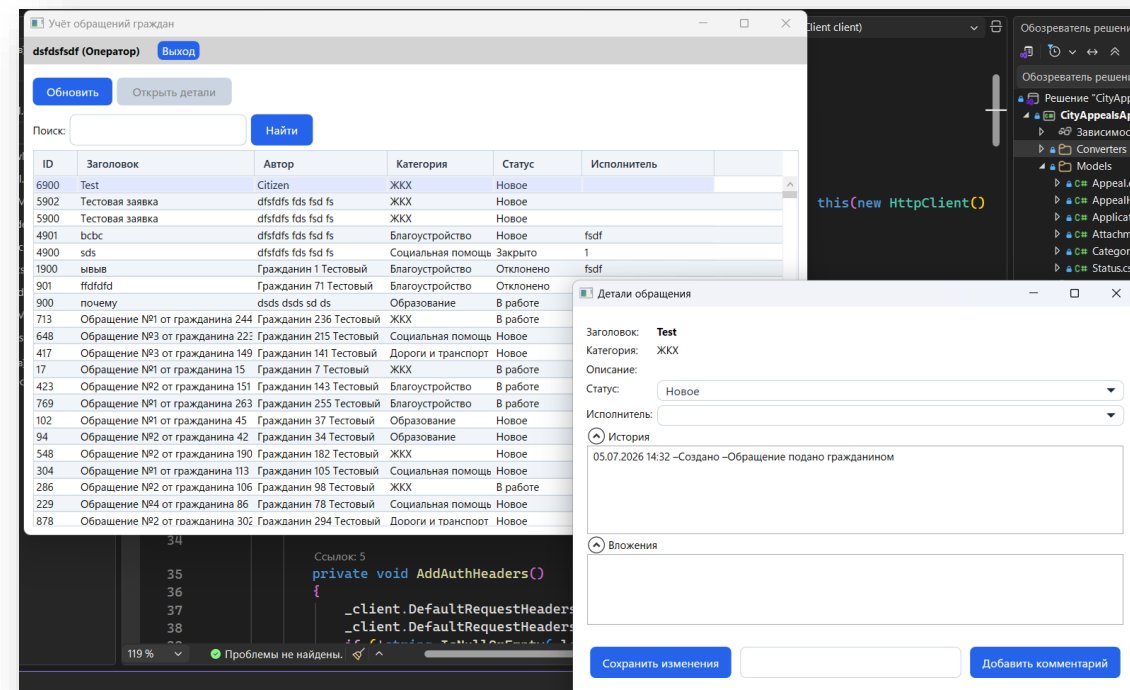
Окно входа
клиентского
приложения

```
D:\study\правктико\произвс x + -
dbug: Microsoft.AspNetCore.Watch.BrowserRefresh.BlazorWasmHotReloadMiddleware[0]
Middleware loaded
dbug: Microsoft.AspNetCore.Watch.BrowserRefresh.BrowserScriptMiddleware[0]
Middleware loaded. Script /_framework/aspnetcore-browser-refresh.js (16513 B).
dbug: Microsoft.AspNetCore.Watch.BrowserRefresh.BrowserScriptMiddleware[0]
Middleware loaded. Script /_framework/blazor-hotreload.js (799 B).
dbug: Microsoft.AspNetCore.Watch.BrowserRefresh.BrowserRefreshMiddleware[0]
Middleware loaded: DOTNET_MODIFIABLE_ASSEMBLIES=debug, __ASPNETCORE_BROWSER_TOOLS=true
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5146
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: D:\study\правктико\производственное\пн02\гит\PM02_PrPr_2026\Файлы\API\CityAppealsApi\CityAppeal
sApi
|
```

Командная строка API ASP.NET

ИНТЕГРАЦИЯ С API

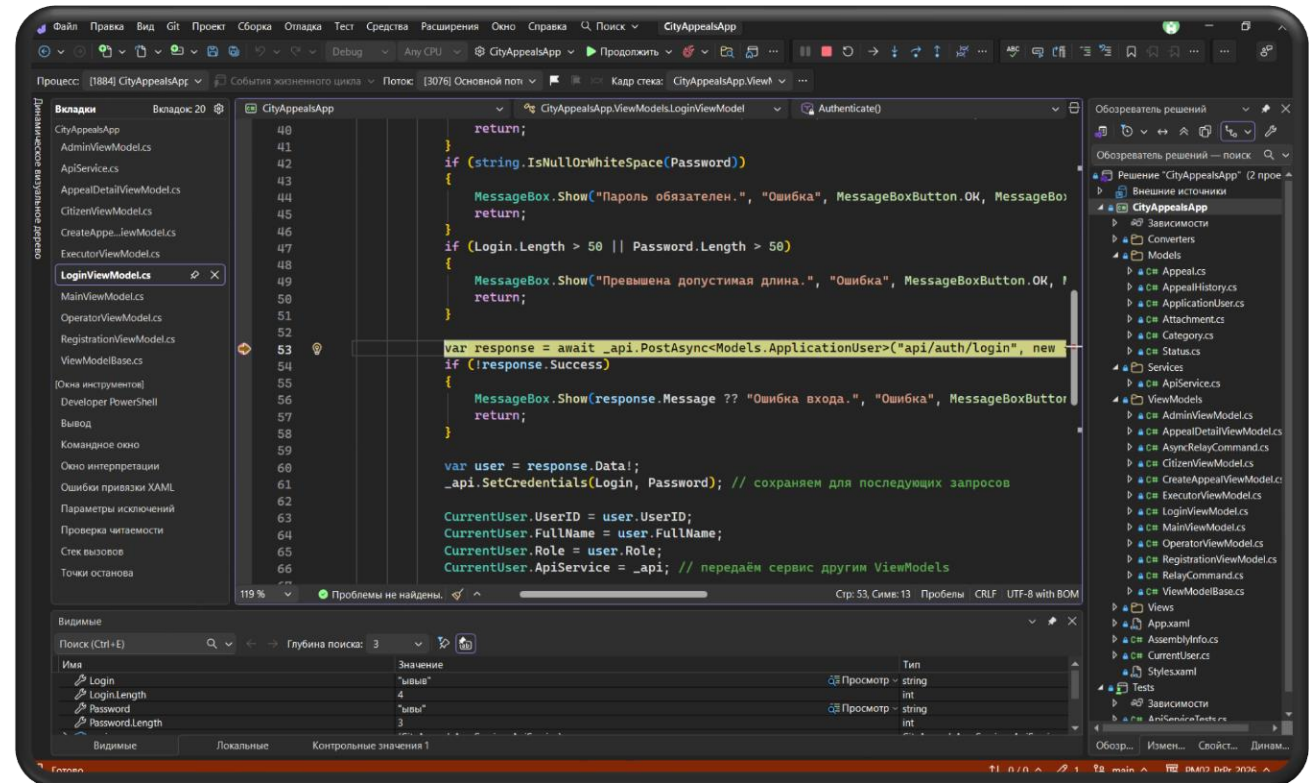
Интеграция выполнена путём замены прямого доступа к базе данных в клиентском приложении на вызовы API. Сервис ApiService формирует HTTP-запросы, добавляет авторизационные заголовки X-Login и X-Password, обрабатывает ответы. При ошибках сервера или сети пользователю отображаются соответствующие сообщения.



Отображение данных, полученных через API

ОТЛАДКА ПРОГРАММНЫХ МОДУЛЕЙ

Отладка проводилась с использованием встроенных средств Visual Studio, а также инструментов тестирования HTTP-запросов (Postman). Для серверной части проверялась корректность маршрутов и ответов, для клиентской – правильность формирования запросов и обработки ответов. Выявленные ошибки, связанные с циклическими ссылками при сериализации JSON, были устранены путём проецирования данных на анонимные типы.



Выполнение отладки в Visual Studio

РЕФАКТОРИНГ ПРОГРАММНОГО КОДА

Доработка обработкой исключительных ситуаций

В клиентском приложении реализована централизованная обработка исключений в классе ApiService. В зависимости от кода состояния HTTP и наличия сообщения от сервера формируется понятное пользователю сообщение. Например, ошибка 401 «Unauthorized» преобразуется в «Неверный логин или пароль, либо недостаточно прав», ошибка 404 – «Запрашиваемый объект не найден». При сетевых сбоях выводится «Ошибка соединения».

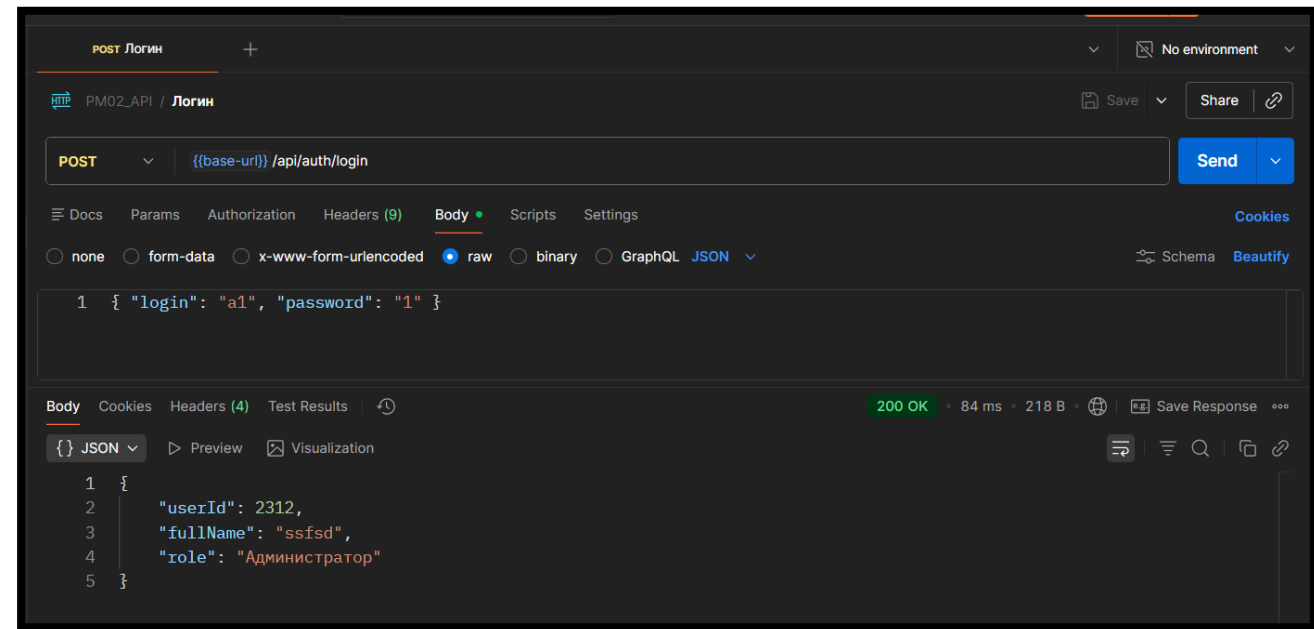
Рефакторинг программного кода

- Изменена архитектура клиента: добавлен слой сервиса ApiService, избавив ViewModel-классы от прямого использования ApplicationDbContext.
- Все команды, связанные с вводом-выводом, переведены на асинхронное выполнение с помощью AsyncRelayCommand, что исключает блокировку пользовательского интерфейса.
- Модели данных адаптированы под JSON-сериализацию.
- Проведена очистка неиспользуемых зависимостей и унификация обработки ответов от API.

РУЧНОЕ ТЕСТИРОВАНИЕ

Ручное тестирование проводилось по заранее составленным тест-кейсам, охватывающим функциональность всех ролей. Всего разработано и выполнено 29 тестов (16 тест-кейсов для клиентского приложения и 13 запросов в Postman для тестирования API).

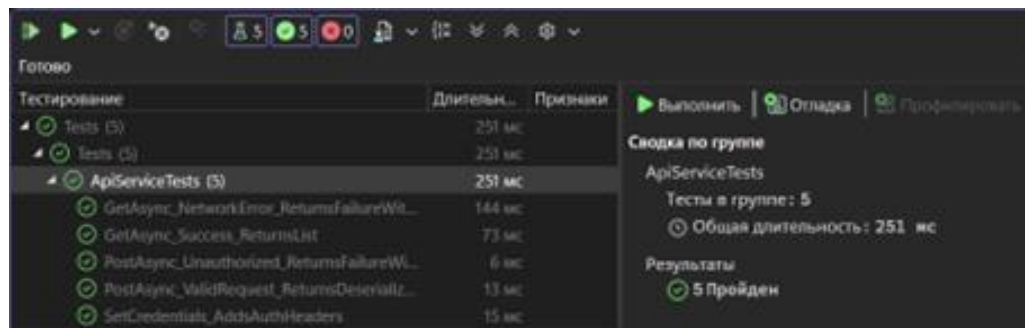
Все тесты завершились успешно, что подтверждает соответствие реализованной функциональности требованиям.



Выполнение тестового запроса в Postman

МОДУЛЬНОЕ И ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ

Модульное тестирование проведено для клиентского сервиса ApiService с использованием фреймворков NUnit и Moq. Были протестированы сценарии успешного выполнения запросов, ошибок аутентификации, сетевых исключений и проверки добавления заголовков.



Готово

Тестирование	Длительн...	Признаки
Tests (5)	251 мс	
Tests (5)	251 мс	
ApiServiceTests (5)	251 мс	
GetAsync_NetworkError_ReturnsFailureWi...	144 мс	
GetAsync_Success_ReturnsList	73 мс	
PostAsync_Unauthorized_ReturnsFailureWi...	6 мс	
PostAsync_ValidRequest_ReturnsDeserializ...	13 мс	
SetCredentials_AddsAuthHeaders	15 мс	

Выполнить | Отладка | Профилировать

Сводка по группе

ApiServiceTests

Тесты в группе: 5

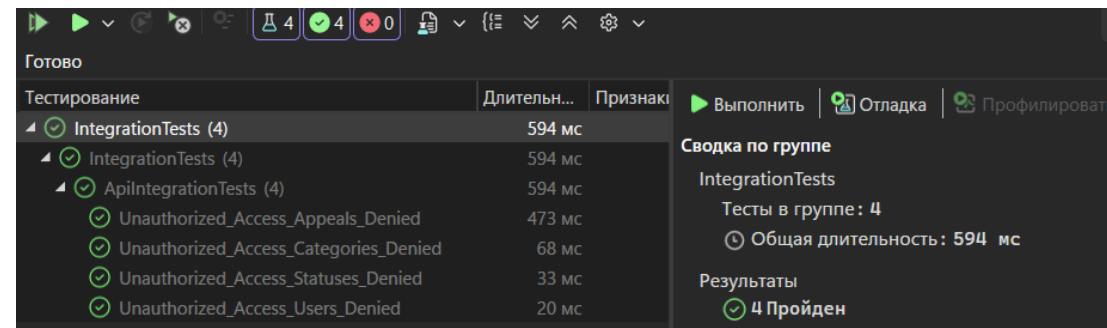
Общая длительность: 251 мс

Результаты

5 Пройден

Результаты модульного тестирования

Интеграционные тесты выполнены для API с помощью WebApplicationFactory и in-memory базы данных. Сценарии включают полный жизненный цикл обращения (регистрация, вход, создание, получение списка, удаление), работу оператора по обновлению статуса и назначению исполнителя, доступ администратора к списку пользователей и проверку отказа в доступе без авторизации.



Готово

Тестирование	Длительн...	Признаки
IntegrationTests (4)	594 мс	
IntegrationTests (4)	594 мс	
ApiIntegrationTests (4)	594 мс	
Unauthorized_Access_Appeals_Denied	473 мс	
Unauthorized_Access_Categories_Denied	68 мс	
Unauthorized_Access_States_Denied	33 мс	
Unauthorized_Access_Users_Denied	20 мс	

Выполнить | Отладка | Профилировать

Сводка по группе

IntegrationTests

Тесты в группе: 4

Общая длительность: 594 мс

Результаты

4 Пройден

Результаты интеграционного тестирования

РАЗРАБОТКА ДОКУМЕНТАЦИИ

Для разработанной программной системы была написана сопровождающая документация:

- 1. Руководство оператора** - описывает порядок работы с приложением для всех категорий пользователей. Документ содержит инструкции по запуску, входу в систему, основным операциям (создание, обработка, удаление обращений, администрирование пользователей) и действиям в нештатных ситуациях.
- 2. Пояснительная записка** - содержит описание архитектуры системы, перечень реализованных функций, обоснование технических решений, результаты тестирования и инструкции по развёртыванию. Документ позволяет получить полное представление о программном комплексе, его структуре и логике работы.

1. Введение

Документ содержит описание программного комплекса «Учёт обращений граждан в администрацию города», разработанного в рамках производственной практики. Комплекс состоит из серверной части (Web API) и клиентского приложения (WPF), взаимодействующих по протоколу HTTP.

2. Назначение системы

Система предназначена для автоматизации процессов:

- Подачи и регистрации обращений граждан.
- Обработки обращений сотрудниками администрации.
- Контроля статусов и сроков исполнения.
- Формирования отчётности.

Целевая аудитория: граждане, операторы приёмной, исполнители (специалисты отделов) и администратор.

3. Архитектура программного модуля

Система построена по двухзвенной архитектуре «клиент – сервер». Серверная часть реализована в виде RESTful API на платформе ASP.NET Core 10, клиентская – в виде настольного приложения Windows Presentation Foundation (WPF) на .NET 10.

Клиент (CityAppealsApp.exe):

- Реализует графический интерфейс пользователя.
- Взаимодействует с API через HTTP-запросы, инкапсулированные в сервисном слое.

Написание пояснительной записки

ЗАКЛЮЧЕНИЕ

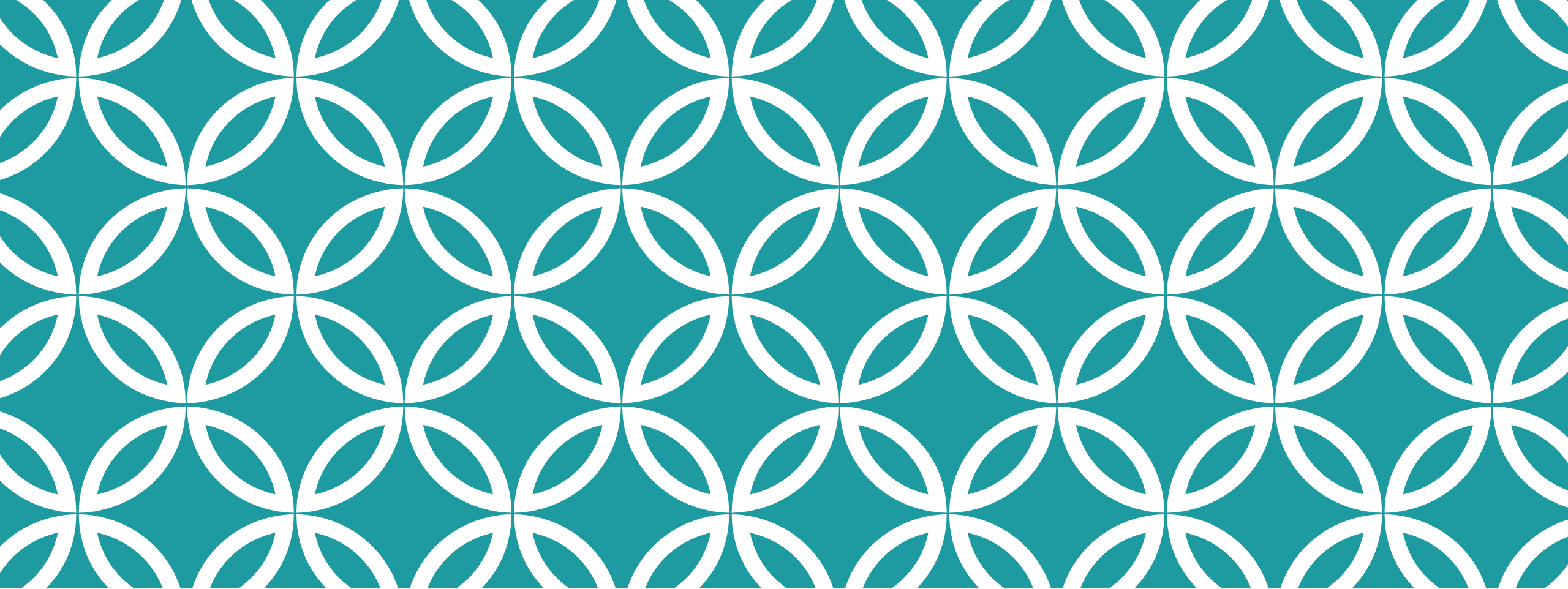
В ходе выполнения практики по модулю «Осуществление интеграции программных модулей» была достигнута поставленная цель – разработана информационная система учёта обращений граждан, состоящая из серверного API и клиентского приложения.

Решены следующие задачи:

- Проведён анализ предметной области, построены UML-диаграммы (Use Case) и ER-модель.
- Спроектирована нормализованная база данных и выполнено её развёртывание с заполнением тестовыми данными.
- Разработан программный модуль в виде Web API на ASP.NET Core и WPF-клиента.
- Реализована интеграция клиента с API, включая авторизацию и обработку ошибок.
- Выполнена отладка, рефакторинг, ручное, модульное и интеграционное тестирование.
- Подготовлены руководство оператора и пояснительная записка.

Полученные навыки проектирования, разработки и тестирования многоуровневых приложений, а также работы с системой контроля версий соответствуют задачам практической подготовки.

Репозиторий с файлами практики - https://github.com/ScioSilentium13/PM02_PrPr_2026



СПАСИБО ЗА ВНИМАНИЕ!